ShopNexus

Multi-Vendor E-commerce Platform

Project Documentation

Student Name	Vivek Das
Roll Number	2305830
Course	Full Stack with Java
Project Title	ShopNexus - Multi-Vendor E-commerce Platform
Repository	https://github.com/harryson22102004/electiveproject
Live Frontend	https://shopnexus-ui.vercel.app

Submitted as a final project report for evaluation.

1. Student Information

Name: Vivek Das

Roll Number: 2305830

Course: Full Stack with Java

Project Title: ShopNexus - Multi-Vendor E-commerce Platform

2. Project Overview

ShopNexus is a full-stack multi-vendor e-commerce application built as an academic project to demonstrate end-to-end web development using Spring Boot and React. The solution includes secure authentication, product browsing, cart management, checkout flow, order creation, validation, and responsive UI components.

The project follows a layered backend architecture with Controller, Service, Repository, Entity, and DTO layers. On the frontend, React components are organized into reusable UI building blocks and route-based pages.

3. Objectives

- Build a complete full-stack application with a clean separation between frontend and backend.
- Demonstrate Spring Boot REST API development with JPA-based persistence.
- Implement JWT-based authentication and role-based support.
- Create a responsive React UI that consumes backend APIs.
- Prepare project-quality documentation and presentation material for submission.

4. Technology Stack

- **Backend:** Spring Boot 3.x, Spring Security, Spring Data JPA, Hibernate Validator, JWT, Maven.
- **Frontend:** React 19, Vite, React Router, Axios, Framer Motion, Tailwind CSS utilities.
- **Database:** MySQL / H2 for development and testing.
- **Testing:** JUnit 5, Mockito, Spring Boot Test, MockMvc.
- **Documentation:** Markdown project report, architecture diagram, ER diagram, Postman collection, and PDF report.

5. Functional Scope

- User registration and login with JWT token generation.
- Public product listing and product details view.
- Cart operations: add, update quantity, remove.
- Checkout and order creation flow.
- Backend validation and error handling.
- Admin/vendor/customer friendly architecture for future expansion.

6. Architecture Summary

The frontend (React + Vite) communicates with the backend using JSON over HTTP. The backend persists data with Spring Data JPA and Hibernate. The system uses DTOs for request and response payloads to keep API contracts clean and maintainable.

Layers: Controller → Service → Repository → Entity, with DTOs used for data exchange and validation between UI and persistence layers.

7. Data Model Summary

Core entities include User, Product, Order, and OrderItem. Users have roles, vendors own products, and customers place orders containing one or more order items. The report includes a Mermaid ER diagram in the repository documentation for easy reference.

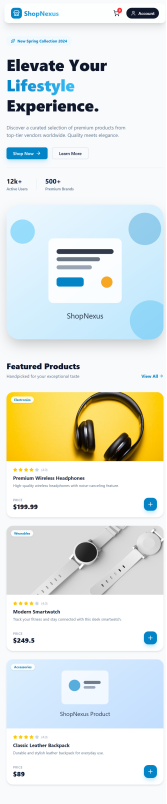
8. Implementation Highlights

- Authentication endpoints: `/api/auth/register` and `/api/auth/login`.
- Product APIs: list all products, get product by ID, create product.
- Order APIs: create order, fetch order by ID, fetch orders by user.
- React pages: Home, Cart, and Checkout.
- Reusable components: Navbar, ProductCard, Cart drawer.
- Local image fallback assets to avoid external image blocking in browsers.

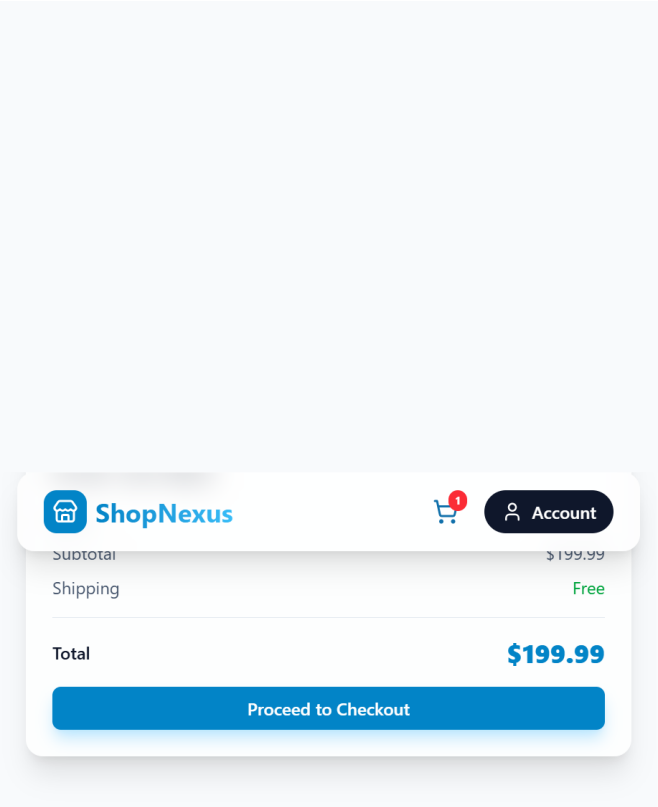
9. Validation and Testing

The backend was verified using integration and unit tests. The frontend build was also validated successfully. The application demonstrates a complete run from the browser UI to backend API responses and order creation.

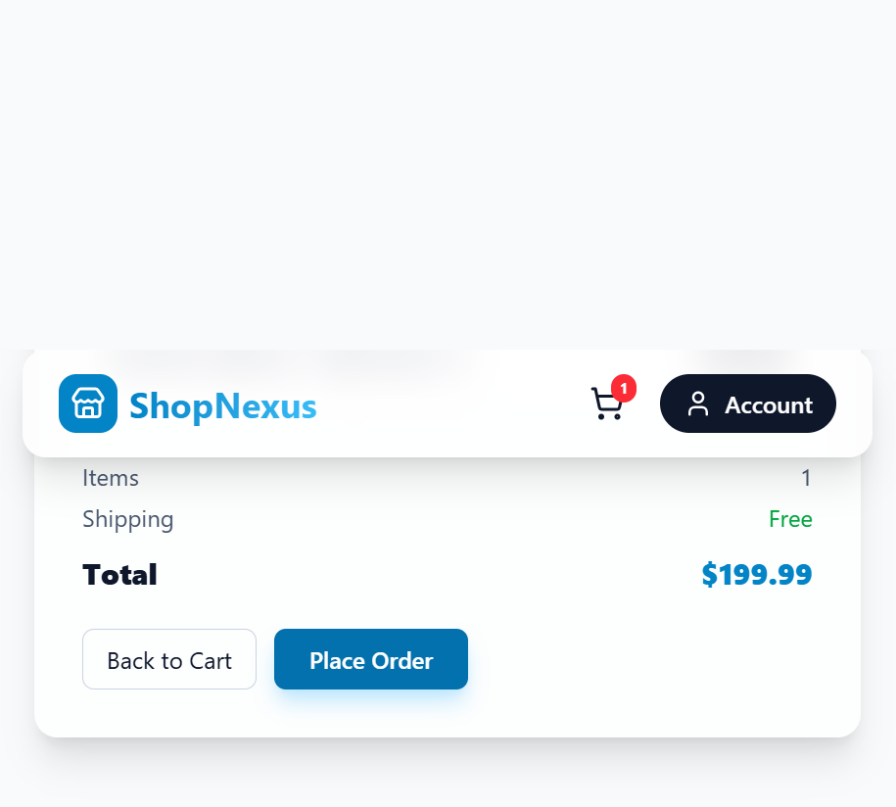
10. UI Screenshots



Home page showing the hero section and product listing.



Cart page showing selected product, quantity controls, and order summary.



Checkout page showing the final order review before placing the order.

11. Limitations & Future Scope

Current limitations include a simplified checkout flow, demo-oriented security policies, static/fallback product media, and the absence of advanced observability dashboards.

Future scope includes stricter JWT filter-based endpoint policies, payment gateway integration, vendor analytics dashboards, cloud image storage, and CI/CD plus monitoring support.

12. Conclusion

ShopNexus demonstrates the complete lifecycle of a modern full-stack web application and is suitable for academic evaluation because it covers architecture, security, API design, UI implementation, validation, testing, and documentation.